COSTRA

Cost-Based SQL Compiler & Query Optimizer

REVIEW 1 + REVIEW 2 + REVIEW 3

Computer Design Laboratory • BCSE307P

Arsh Shukla | 23BCE10043 | VIT Vellore

github.com/arshshukla500newuser/COSTRA-SQL-Compiler

Combined report covering project design, implementation, optimization, validation, and GitHub update steps.

1. Review overview

Review 1 establishes the problem, objectives, scope and architecture. Review 2 demonstrates lexer, parser/AST, semantic validation and relational IR. Review 3 presents candidate-plan costing, selection and SQLite execution validation.

2. Problem and objectives

SQL describes the result, but scans, filters, joins and aggregation can be arranged in different ways. COSTRA makes a supported SQL subset inspectable by tokenizing and parsing queries, validating schema references, building relational IR, estimating cardinality/cost and comparing plans. It is an educational prototype, not a production optimizer.

3. System architecture

SQL input → Lexer → Parser/AST → Semantic analysis → Relational IR → Cardinality/selectivity → Candidate plans → Cost comparison → Normalized SQL → SQLite results.

Modules: main.py (workflow), lexer.py (tokens), parser.py and ast_nodes.py (grammar/AST), semantic.py and schema.py (validation/schema), ir.py (logical plan), statistics.py and optimizer.py (estimation/selection), executor.py (SQLite), test_costra.py (unit tests).

4. Compiler Design connection

Lexical analysis converts characters into tokens. Syntax analysis checks the supported grammar and builds an AST. Semantic analysis resolves aliases and checks columns. Relational IR represents logical operations independently of SQL text. Optimization compares estimated candidate costs. SQLite execution validates the normalized SQL. COSTRA uses relational IR, not three-address code (TAC).

5. Schema and semantic analysis

Project schema: Employee(id, name, salary, dept_id, experience); Department(id, name); Project(id, name, budget, manager_id, dept_id); Assignment(employee_id, project_id, hours).

Valid basic-query aliases: e → Employee, d → Department. Unknown columns must be rejected before planning/execution.

6. Review 2 — Lexer, parser and AST

The lexer recognizes SQL keywords, identifiers, literals, operators and punctuation. The parser checks the supported SQL grammar and creates an AST representing SELECT items, FROM/JOIN, WHERE, GROUP BY, HAVING, ORDER BY and LIMIT where supported. The AST lets later stages inspect query structure without reparsing raw SQL.

7. Review 2 — Relational IR

Example logical plan:

```
PROJECT [e.name, d.name]
  FILTER [(e.salary > 50000)]
    JOIN [INNER ON (e.dept_id = d.id)]
      TABLE_SCAN [Employee AS e]
      TABLE_SCAN [Department AS d]
```

PROJECT chooses output expressions, FILTER applies WHERE, JOIN combines relations and TABLE_SCAN represents base relations.

8. Review 3 — Optimization workflow

```
1. Build supported access-path alternatives.
2. Explore bounded join-order alternatives.
3. Estimate intermediate cardinalities.
4. Calculate relative cost for each candidate.
5. Compare costs and mark minimum-cost candidate(s).
```

Join strategies discussed: nested-loop, hash and merge joins. The prototype does not claim full dynamic-programming enumeration.

9. Statistics and cost model

Configured synthetic row counts: Employee 10,000; Department 4; Project 1,200; Assignment 30,000. The earlier report describes illustrative relative formulas: table scan = estimated rows × 2.0; index scan = estimated rows × 0.35 × 2.0; hash join = (left + right rows) × 1.5; merge join = (left + right rows) × 1.8; nested-loop = left rows × right rows; filter adds input row count; sort/aggregate = rows × 1.2 / rows × 1.1. These are version-specific model units, not milliseconds. Use local run output.

10. Review 3 — Candidate cost table

Captured successful custom-query run:

```
Candidate plan | Estimated cost | Estimated rows | Status
Plan 1 | 10,030.58 | 8,000.00 | SELECTED
Plan 2 | 10,030.58 | 8,000.00 | TIED FOR MINIMUM
```

Lowest estimated cost: 10,030.58 model units. Two candidates were evaluated. These figures describe that captured run, not all queries.

11. Test 1 — Basic JOIN + WHERE

```
SELECT e.name, d.name
FROM Employee e
JOIN Department d ON e.dept_id = d.id
WHERE e.salary > 50000;
```

Captured SQLite output: Rahul—Sales; Kabir—Sales; Priya—HR; Neha—Engineering; Aman—Engineering; Arjun—Engineering; Ishita—Finance. Rows returned: 7. The same capture showed two candidates tied at 10,030.58.

12. Test 2 — Multi-join aggregation

```
SELECT d.name AS department, COUNT(e.id) AS employee_count,
       AVG(e.salary) AS average_salary
FROM Employee e
JOIN Department d ON e.dept_id = d.id
JOIN Assignment a ON e.id = a.employee_id
JOIN Project p ON a.project_id = p.id
WHERE e.salary > 60000 AND p.budget > 100000
GROUP BY d.name
HAVING AVG(e.salary) > 65000
ORDER BY AVG(e.salary) DESC
LIMIT 5;
```

Demonstrates multiple joins, COUNT/AVG, GROUP BY, HAVING, ORDER BY and LIMIT. Record the actual current-run costs and results; historical outputs may differ by version.

13. Test 3 — Access path, ORDER BY and LIMIT

```
SELECT e.name, e.salary
FROM Employee e
WHERE e.salary > 80000
ORDER BY e.salary DESC
LIMIT 5;
```

Demonstrates filtering, possible salary-index costing, sorting and limiting. Capture the candidates and results generated by the local version; do not assume a particular candidate count.

14. Negative test — Semantic error

```
SELECT e.unknown_column
FROM Employee e;
```

Expected: syntax may pass, but semantic analysis reports an unknown column and stops before candidate-plan generation/execution. This shows syntax validity and semantic validity are separate checks.

15. COSTRA vs SQLite

COSTRA builds IR, estimates cardinality and candidate costs, selects the minimum estimate and prints plan analysis. SQLite independently chooses its internal plan and executes normalized SQL. The selected educational COSTRA plan is not injected into SQLite.

16. Validation summary

Capture evidence for: tokens and AST; semantic alias map and invalid-column diagnostic; relational IR; candidate cost table and selection status; normalized SQL and actual result rows; advanced multi-join query if supported. This is a list of observable demo evidence, not an unverified automated-test count.

17. Limitations and future work

Limitations: supported SQL subset only; synthetic statistics and costs; bounded join-order exploration; no plan injection into SQLite; local timings are not general benchmarks.
Future work: more edge-case tests, clearer plan/tie explanations, repeatable benchmark fixtures and README instructions aligned with the current branch.

18. Run the demo

From the repository directory run: python3 main.py. Choose the integrated workflow and either the built-in query or custom SQL. Optional commands such as python3 main.py --demo or python3 -m unittest should be used only if supported by the current local version.

19. Update GitHub

After copying the PDF and PPTX into docs/reviews/, run:

```
git status
git branch --show-current
git pull --rebase origin main
git add docs/reviews/COSTRA_Review_1_2_3_Report.pdf docs/reviews/COSTRA_Review_1_2_3_Presentation.pptx
git commit -m "Add combined Review 1 2 3 report and presentation"
git push origin main
```

Use the actual branch if it is not main. Review staged files and avoid committing secrets or environment files.

20. Conclusion

Review 1: design foundation. Review 2: compiler implementation. Review 3: optimization and execution validation. COSTRA makes SQL processing and cost-based reasoning inspectable while clearly distinguishing synthetic estimates from actual SQLite execution.

Arsh Shukla | 24BDS0113 | VIT Vellore

Thank You!